

GEOMATRIX — Documentation & File Structure

Smart India Hackathon 2026 · Comprehensive Project Overview

1. Frontend: Next.js Web Application

- **app/**: Main frontend routing directory.
- **app/dashboard/page.tsx**: The Command Center. Provides live KPI metrics, AI risk score charts, and real-time acquisition pipeline statuses.
- **app/projects/[id]/page.tsx**: Deep-dive Risk Analysis page for a single project. Contains What-If simulator and Gemini AI mitigations.
- **app/map/page.tsx**: GIS Risk Map. Plots projects using latitude/longitude and color codes them by risk level.
- **app/data/page.tsx**: Data Management interface. Allows uploading MoSPI CSV files for training and prediction.
- **lib/apiClient.ts**: Centralized API client connecting the frontend to the FastAPI backend.

2. Backend: FastAPI (geomatrix_v2)

- **main.py**: Entry point for the FastAPI server.
- **models.py**: Defines SQLAlchemy database schemas for Projects, Alerts, ModelRuns, etc.
- **routers/**: Contains API route handlers.
- **routers/projects.py**: CRUD endpoints for projects, dashboard stats, and GIS geojson data.
- **routers/model.py**: Endpoints to train the ML model (RandomForest) and fetch model status.
- **routers/ingest.py**: CSV ingestion logic handling historical data and live project data.
- **routers/gemini.py**: API endpoint for generating AI mitigation strategies via the Gemini LLM.

3. Database (geomatrix.db)

- **Database Type**: SQLite database (geomatrix.db) located in the backend folder.
- **projects table**: Stores all live project data (land acquired, legal cases, etc.) along with ML risk scores and delay probabilities.
- **historical_delay_records table**: Stores past project data used to train the ML model.
- **model_runs table**: Logs each ML training session with its precision, recall, and ROC-AUC metrics.
- **alerts table**: Stores early warning notifications for critical/high-risk projects.

4. Machine Learning & Execution Scripts

- **seed_db.py**: Automates the population of the database with initial projects and historical records.
- **ml/risk_model.py** (or similar): Defines the Random Forest pipeline and data scaling logic for risk prediction.
- **execute_tasks.py**: A script in the scratch folder used to batch-ingest data and train the model end-to-end.
- **audit_risk.py / batch_predict.py**: Utility scripts to audit missing risk scores and batch-predict all projects in the DB.